

USED MEDIAS LLC

EMBEDDED SYSTEMS DIVISION

# Engineering Documentation

nat-os

| DOCUMENT | REVISION | STATUS | CLASSIFICATION |
|----------|----------|--------|----------------|
| nat-os   | —        | —      | Internal       |

Hardware: developed and verified on the ESP32-2432S028R ("Cheap Yellow Display")

Renamed from `cyd-os` . Document numbering and section numbering are unchanged — [UM-CYDOS-014 §5.2](#) in an older commit message is [UM-NATOS-014 §5.2](#) here. The rename reflects that only the pin maps and the panel and touch drivers are specific to the Cheap Yellow Display; everything above them assumes an ESP32 and nothing more.

## Purpose of this set

These reports record the design of nat-os in enough detail that the project can be picked up cold — by someone else, or by the author after six months away — without re-deriving decisions from the source.

Each report states **what was decided, what it was measured against, and what remains unverified**. Claims that were tested on hardware are marked as such. Claims taken from documentation or reasoning are marked separately, because on a from-scratch kernel the difference between "this is true" and "this should be true" is the difference between a working boot and a silent reboot.

## Index

| ID           | Title                                    | Covers                                                                                                                                                           |
|--------------|------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| UM-NATOS-001 | System Architecture and Scope            | Layer model, what is built vs borrowed, the isolation problem, why a bytecode VM                                                                                 |
| UM-NATOS-002 | Boot Chain and Image Format              | ROM → 2nd stage → kernel, image header, measured boot trace                                                                                                      |
| UM-NATOS-003 | Xtensa ABI Selection                     | Windowed vs call0, codegen evidence, consequences for the scheduler                                                                                              |
| UM-NATOS-004 | Memory Map and Allocation Policy         | ESP32 regions, our layout, the bootloader overlap risk                                                                                                           |
| UM-NATOS-005 | Build and Flash Pipeline                 | Toolchain, flags and why each one, reproducing a build, why not PlatformIO                                                                                       |
| UM-NATOS-006 | Milestone 0 Verification Report          | Test method, captured output, pass/fail per assertion                                                                                                            |
| UM-NATOS-007 | Development Roadmap M1–M5                | Each milestone, its risks, and its exit criteria                                                                                                                 |
| UM-NATOS-008 | Milestone 1 Verification Report          | Vectors, timer source and level choice, interrupt entry/exit, measured results, and a second writer to the comparator that stalled the tick for 183 ms           |
| UM-NATOS-009 | Milestone 2 Verification Report          | Task model, context frame, scheduler, the zero-overhead <code>LOOP</code> defect, watchdog correction, and ageing to bound the starvation strict priority allows |
| UM-NATOS-010 | Milestone 3 Verification Report          | Heap allocator, arena model and bounds checking, and the measured DRAM budget                                                                                    |
| UM-NATOS-011 | Flash Cache and Read-Only Data Placement | Mapping <code>.rodata</code> into flash, the 0x20 page congruence, and the cache-off hazard                                                                      |
| UM-NATOS-012 | Milestone 4 Verification Report          | Register-based bytecode VM, the ISA, the assembler, containment of malformed programs, and the syscalls added since                                              |

| ID           | Title                                                            | Covers                                                                                                                                                                                                                                                                                       |
|--------------|------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| UM-NATOS-013 | Milestone 5 Verification Report                                  | Application table and lifecycle, three-level scheduling, the shell, the escape attempt, and messaging without shared memory                                                                                                                                                                  |
| UM-NATOS-014 | Locking Primitives                                               | Critical sections vs blocking mutex, task blocking and idle, a measured starvation defect, and why contention cost is the number of blocking events rather than the time held                                                                                                                |
| UM-NATOS-015 | Display Driver                                                   | ILI9341, span rendering with no framebuffer, and the path from bit-banged SPI to SPI2 with DMA                                                                                                                                                                                               |
| UM-NATOS-016 | Display Syscalls, and a Total System Freeze                      | Per-application viewports, pointer discipline, measured containment, and a yield that stopped the clock                                                                                                                                                                                      |
| UM-NATOS-017 | Touchscreen, and a Verification Method That Failed Three Times   | XPT2046 over PENIRQ, the GPIO two-bank bug, calibration by controlled input, why the four corners were the wrong four points, a capture that erased its own evidence, input confinement, and an axis that was inverted for three months because the direction test read its own worst sample |
| UM-NATOS-018 | Persistence, and a Read Defect That Looked Like the Wrong Thing  | SPI flash driver, a checksummed record that survives a power cycle, the inherited clock divider that shifted every read by a bit, and two hypotheses tested against stale firmware                                                                                                           |
| UM-NATOS-019 | Failure Handling, and Three Mechanisms That Had Never Fired      | Stack guards enforced in the scheduler, the watchdog erasing its own panic reports, measured stack margins, a UART receive path one byte behind, and a fault reported to flash and to the panel                                                                                              |
| UM-NATOS-020 | microSD over SPI, and a Pad Table That Is Not in Pin Order       | SPI mode and why, per-stage error codes, the IO_MUX entry that is the UART's receive pad, and a cross-check whose samples could not have caught it                                                                                                                                           |
| UM-NATOS-021 | The Launcher, and Four Defects It Found by Existing              | Icon grid and hybrid cursor, launch by name, status text that lied about state, selection read at the worst moment, the missing idle task, a close button an application cannot reach, and a correct diagnosis fixed at twenty times the necessary scope                                     |
| UM-NATOS-022 | The Note Pad, and a Workaround Wearing a Costume                 | Multi-tap keypad, messages saved to flash and read back after a power cycle, and a keypad whose key size was a touch calibration fault in disguise, plus what that fault actually was                                                                                                        |
| UM-NATOS-023 | The Interrupt Matrix, and Four Ways to Deliver an Edge to Nobody | The first peripheral interrupt this kernel routes: sources vs lines, a shared vector that could only have been right by accident, a PRO CPU enable bit that had to be measured, and four failures in which an edge was delivered perfectly to something that could not act on it             |
| UM-NATOS-024 | The ADC, and a Wrong Bit in a Right Register                     | SAR ADC1, the board's light sensor confirmed rather than assumed, and a one-bit error invisible to every read-back because it sat inside a twelve-bit field                                                                                                                                  |
| UM-NATOS-    | I2C, and Why a Preempted Master Is                               | Bit-banged two-wire master on the last two free pins, clock stretching as the reason preemption is safe, and a self-test for the failure where a missing pull-                                                                                                                               |

| ID                   | Title                                           | Covers                                                                                                                                                                                   |
|----------------------|-------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 025                  | Legal                                           | up makes every address answer                                                                                                                                                            |
| UM-<br>NATOS-<br>026 | The Shell on the Panel,<br>and Not a Menu of It | The real shell reached from a multi-tap keypad rather than a second command set that would drift from it; output captured by teeing the UART for exactly one command                     |
| UM-<br>NATOS-<br>027 | Audio, and Three Ways to<br>Be Silent           | Hardware PWM on the speaker pin, a click on every keypress, and three stacked faults where each one made the next invisible — including a self-check that could not fail by construction |

## Reading order

New to the project: **001** → **002** → **004** → **003**. That gives the shape of the system, how it starts, where things live, and why the calling convention is unusual.

Picking up implementation work: **006** (what is known-good) then **007** (what is next and what it will break).

Reproducing a build: **005** alone is sufficient.

## Status summary as of this revision

| Item                                         | State                                                                                                                                                                                                            |
|----------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Milestone 0 — kernel boots, self-checks pass | <b>Complete, verified on hardware</b>                                                                                                                                                                            |
| Milestone 1 — timer interrupt, tick counter  | <b>Complete, verified on hardware</b>                                                                                                                                                                            |
| Milestone 2 — native task switching          | <b>Complete, verified on hardware</b> — 3,400+ switches, zero corruption                                                                                                                                         |
| Milestone 3 — heap and arena model           | <b>Complete, verified on hardware</b> — all three exit criteria                                                                                                                                                  |
| Milestone 4 — bytecode interpreter           | <b>Complete, verified on hardware</b> — program runs, six fault classes contained                                                                                                                                |
| Isolation                                    | <b>Live</b> — every VM memory access bounds-checked, UM-NATOS-012 §6.2                                                                                                                                           |
| Full stack                                   | <b>Running</b> — bytecode hosted in a preemptible native task, 3.3M instructions across 450 preemptions with zero accounting drift, UM-NATOS-012 §6.6                                                            |
| Milestone 5 — multiple applications          | <b>Complete, verified on hardware</b> — all three exit criteria                                                                                                                                                  |
| Roadmap M0–M5                                | <b>Complete.</b> Six native tasks, three scheduling levels, a shell, and an application deliberately written to escape its arena that could not                                                                  |
| Locking                                      | <b>Complete</b> — critical sections and a blocking mutex; heap and console both arbitrated, UM-NATOS-014                                                                                                         |
| Task blocking                                | <b>Live</b> — <code>TASK_BLOCKED</code> , <code>TASK_SLEEPING</code> and an idle task using <code>WAITI</code> , UM-NATOS-009 §11                                                                                |
| Scheduling                                   | <b>Priorities with ageing</b> — three levels plus ageing credit, so a ready task reaches the front within ~600 ms whatever sits above it; <code>task_sleep()</code> and priority inheritance, UM-NATOS-009 §11.4 |
| 3D renderer                                  | <b>Running</b> — grid raycaster, one ray per column, face shading, framebuffer on (§5.7's contrary measurement was taken through a stalling clock and lock contention), UM-NATOS-015 §5.8                        |
| Display                                      | <b>Working on hardware</b> — ILI9341, no framebuffer, <b>43 ms</b> full-screen fill via SPI2 + DMA, UM-NATOS-015 §5.5                                                                                            |
| Application graphics                         | <b>Live</b> — <code>FILL / TEXT / DIMS</code> confined to per-application viewports; 0 escapes across 136 audited fills, UM-NATOS-016 §5                                                                         |
| Touch                                        | <b>Working on hardware</b> — XPT2046 gated on PENIRQ and pressure; X axis was inverted for three months; calibrated on-device from four inset targets and persisted across reset, UM-NATOS-017 §4.1, §7.4        |

| Item                  | State                                                                                                                                                                                                                               |
|-----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Interrupts            | <b>Matrix verified, no consumer</b> — peripheral interrupts routable for the first time; PENIRQ proven by injection but never observed to fire from a finger, so touch stays polled, UM-NATOS-023 §6, §7                            |
| ADC                   | <b>Working on hardware</b> — SAR ADC1, 8 channels; the light sensor on GPIO34 moved 265 counts against a 47-count control group, UM-NATOS-024 §6                                                                                    |
| I2C                   | <b>Bus verified, nothing attached</b> — bit-banged on GPIO22/27; both lines drive and release correctly, no byte has yet been transferred, UM-NATOS-025 §7, §8                                                                      |
| On-screen shell       | <b>Working on hardware</b> — all 25 commands reachable with no host attached; one command set, not two, UM-NATOS-026 §2                                                                                                             |
| Audio                 | <b>Working on hardware</b> — LEDC hardware PWM on GPIO26, no CPU while sounding; a click on every keypress. Tones only, and 440 Hz is inaudible on this speaker while 3 kHz is clear, UM-NATOS-027 §4                               |
| Application input     | <b>Live</b> — <code>SYS TOUCH</code> confined to the asking application's viewport; 81 delivered, 109,211 withheld, 0 confinement failures, UM-NATOS-017 §8                                                                         |
| Application messaging | <b>Live</b> — copied through a kernel mailbox, never shared memory; 278 sent / 277 delivered / 0 bad buffers, UM-NATOS-013 §8                                                                                                       |
| Application bitmaps   | <b>Live</b> — <code>SYS BLIT</code> , arena-bounded source and viewport-clipped destination, UM-NATOS-012 §10                                                                                                                       |
| DRAM budget           | <b>Measured at M3</b> — 167,680 B allocatable then, 158,000 B after <code>TASK_MAX</code> rose to 8; it has since risen to 12 and three drivers have been added, so the current figure is lower and unremeasured, UM-NATOS-010 §7.2 |
| Flash cache           | <b>Enabled</b> — <code>.rodata</code> mapped from flash, UM-NATOS-011                                                                                                                                                               |
| Persistence           | <b>Live</b> — checksummed record in a flash sector at 2 MB; boot counter and a cumulative frame count survived 16 resets, UM-NATOS-018 §6                                                                                           |
| Failure handling      | <b>Enforced</b> — guards checked on every switch across every task slot; <code>hang</code> / <code>fault</code> / <code>smash</code> each trigger their path on demand, UM-NATOS-019 §5                                             |
| Fault reporting       | <b>Three ways</b> — flash record read back by the next boot, UART report, and the reason drawn on the panel; ordered by decreasing reliability, UM-NATOS-019 §6–7                                                                   |
| microSD               | <b>Reading</b> — SPI mode, per-stage errors, bounded on an empty slot; FAT16 header read at LBA 240, UM-NATOS-020 §5.2                                                                                                              |
| Launcher              | <b>Live</b> — 3×3 icon grid including an on-screen shell, hybrid cursor, double-tap to launch by name; close button per program in a column outside every viewport, so a program cannot hide its own exit, UM-NATOS-021 §6.2        |
| Notes                 | <b>Live</b> — multi-tap keypad, 8 messages of 160 characters in flash, verified across a power cycle and a reflash, UM-NATOS-022 §6                                                                                                 |
| Stack margins         | <b>Measured</b> — worst task uses 444 B of 2,048; minimum margin 78%, UM-NATOS-019 §3.1                                                                                                                                             |
| Version control       | <b>Initialised 2026-08-14</b>                                                                                                                                                                                                       |

| Item                    | State                                                                                                      |
|-------------------------|------------------------------------------------------------------------------------------------------------|
| JTAG debug probe        | Ordered, not in hand                                                                                       |
| Bootloader IRAM overlap | <b>Closed</b> — investigated and disproved, UM-NATOS-004 §5                                                |
| Panic handler           | <b>Closed</b> — exercised deliberately with an <code>i11</code> instruction; prints EXCCAUSE/EPC and halts |
| Watchdogs               | <b>Closed</b> — measured armed, now disabled and read back, UM-NATOS-009 §8                                |

**Standing rule for interrupt handlers — clear `PS.EXCM` before calling C.** Hardware sets `EXCM` on interrupt entry, and while it is set the Xtensa zero-overhead loop-back is disabled: a hardware loop body executes once and falls through to whatever sits at `LEND`. GCC emits these loops for ordinary counted C loops, so this silently degrades **every C function reachable from a handler** — drivers and the VM interpreter included, not just the scheduler where it was found. `_handler_level3` now clears it; any future handler must do the same. UM-NATOS-009 §6.

**Standing rule for shared registers — a hardware register with a software shadow has one safe shape: either one writer, or every writer maintains the shadow.** `timer.c` kept `g_next` as its idea of the comparator deadline while `task_yield()` wrote `CCOMPARE1` directly. The handler then added a whole interval to a deadline that was only 64 cycles old, so every yield pushed the tick further out — 18 tick periods, 183 ms, before anything noticed. The rule below was obeyed exactly and did not prevent it, because it constrains the WRITER and the defect was in the other party's bookkeeping. UM-NATOS-008 §8.

**Standing rule for the scheduler — a yield must never defer the clock it depends on.** `task_yield()` originally wrote `CCOMPARE1 = ccount + 64` unconditionally, so any loop yielding faster than 64 cycles pushed the deadline ahead of `CCOUNT` forever, the timer interrupt stopped firing, and the whole kernel froze — the tick is what drives every context switch. Any routine adjusting a scheduler deadline must only ever move it **earlier**. UM-NATOS-016 §3.

**Standing rule for verification — when an instrument reports its own reading invalid, that outranks every value printed beside it.** Three times across two sessions a frozen marker, a sample counter stuck at an identical value, and a boot banner in a serial capture each said "this measurement is not what you think", and the plausible-looking numbers next to them were believed anyway. Two conclusions were published and later retracted as a result. Latch the quantity so timing cannot lie about it, then feed the system a controlled input rather than interpreting an uncontrolled one. UM-NATOS-017 §6.

**Standing rule for debugging — a negative result is only informative if the experiment demonstrably ran.** Two flash hypotheses were recorded as tested against a board that had never been reflashed: `build.ps1` builds, but `build.ps1 -Flash` is what flashes, and the invocation used named a script that does not exist. Every hypothesis returned bit-identical output, which was read as "none of these are the cause" when it meant "no experiment has run yet". The signature to watch for is **a run of results that do not vary when the input does** — suspect the harness before the theory, and verify the change reached the target rather than inferring it from the absence of an error. UM-NATOS-018 §5.

**Standing rule for verification — a startup artefact is not evidence the thing it introduces works.** The shell was signed off because its banner printed; the banner proves a task was created and the TRANSMIT path works, and says nothing about receive. The receive path was one byte behind for the shell's entire existence, so pressing Enter did nothing until the next keystroke — and every automated test drove it with CR and LF, the one input shape that hides it. Stack guards and the panic handler went unexercised for the same reason: each was confirmed to EXIST rather than observed to WORK. Trigger the mechanism on purpose, or treat it as untested. UM-NATOS-019 §4.1.

**Standing rule for calibration — never infer direction from the endpoint of a gesture.** The first and last samples of a drag are its two least trustworthy, because both sit at a contact transition where the panel is not bridged and the ADC reads its rail. A rail reading is near the top of the range, so a drag ending anywhere "ends near its maximum" and EVERY axis appears to increase — the test can only return one answer. The touch X axis was backwards for three months behind exactly that. Calibrate from labelled points, each a known position paired with a reading, and let direction fall out of comparing labels. UM-NATOS-017 §7.1.

**Standing rule for lock contention — the cost is the NUMBER of blocking events, not the time the lock is held.** Each one costs a scheduling round-trip whether the lock was held for 24 ms or 24 µs: the blocked task is descheduled and must be selected again. Measured here at 63 ms per contention against a 24 ms hold, with the lock free 88% of the time while two tasks sat blocked on it. Narrowing the hold by 25% changed the outcome by nothing. The levers are batching (fewer takes) or not blocking at all ( `try_lock` ) — shortening holds, the intuitive move, does nothing. UM-NATOS-014 §10.5.



**Standing rule for cross-checks — a check only tests what its samples can distinguish.** The SD pin map was verified against four IO\_MUX entries already in the tree. All four confirmed the indexing, and all four sat below the anomaly, so every one of them agreed with the wrong answer: GPIO23 was read from the UART's receive pad. The check was real, it was performed, and it could not have failed. Samples must STRADDLE the thing being verified, not merely agree with it. UM-NATOS-020 §4.2.

**Standing rule for fixes — a correct diagnosis does not license a fix of arbitrary scope.** Chrome drawn over a view that repaints every pixel every frame will strobe; that diagnosis was reached twice and was right both times. The first fix reserved rows for it, which moved the region boundary, the application strips, the colour strip and the grid — four files of constants — and produced a screen that looked wrong for reasons never found, while every measurement insisted the renderer was correct. The second wrote 324 pixels into a buffer that was already being sent. Prefer the fix whose blast radius matches the defect. UM-NATOS-021 §6.6.

**Standing rule for workarounds — tolerating a defect is not fixing it.** The note pad's keys are 80 px because the touch mapping reads about 24 px low on X; a 24 px key was destroyed by that error and an 80 px key absorbs it. The app works, the fault is untouched, and every future element finer than 80 px will meet it again. Record which one you did, at the place a reader would otherwise assume the generous version. UM-NATOS-022 §3.3.

**Standing rule for reverts — reverting two changes together destroys the information about which one mattered.** A relay layout and a camera fix were reverted in one commit; the screen looked right afterwards, which appeared to convict the layout. Re-applying the geometry alone, later, showed it was innocent — the camera had broken concurrently and "blank screen" was a correct rendering of the inside of a wall. If a revert must bundle, the bundle is a hypothesis to test later, not a conclusion. UM-NATOS-021 §6.7.